Loan Default Prediction via Decision Trees

Cameron Batts

```
library(tidyverse)
```

```
## ── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0 ──
## ✔ dplyr     1.1.4     ✔ readr     2.1.5
## ✔ forcats   1.0.1     ✔ stringr   1.5.2
## ✔ ggplot2   4.0.0     ✔ tibble    3.3.0
## ✔ lubridate 1.9.4     ✔ tidyr     1.3.1
## ✔ purrr     1.2.1
## ── Conflicts ────────────────────────────────────────── tidyverse_conflicts() ──
## ✖ dplyr::filter() masks stats::filter()
## ✖ dplyr::lag()    masks stats::lag()
## ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflic
ts to become errors
```

```
library(readr)
```

```
loanData = readRDS("LoanData.rds")
```

```
glimpse(loanData)
```

```
## Rows: 29,092
## Columns: 8
## $ isLoanDefault   <int> 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1, …
## $ loanAmount      <int> 5000, 2400, 10000, 5000, 3000, 12000, 9000, 3000, 1000…
## $ interestRate    <dbl> 10.65, NA, 13.49, NA, NA, 12.69, 13.49, 9.91, 10.65, 1…
## $ creditGrade     <fct> B, C, C, A, E, B, C, B, B, D, C, A, B, A, B, B, B, B, …
## $ employmentYears <int> 10, 25, 13, 3, 9, 11, 0, 3, 3, 0, 4, 13, 1, 6, 17, 13,…
## $ homeLiving      <fct> RENT, RENT, RENT, RENT, RENT, OWN, RENT, RENT, RENT, R…
## $ incomeAnnual    <dbl> 24000.00, 12252.00, 49200.00, 36000.00, 48000.00, 7500…
## $ ageYears        <int> 33, 31, 24, 39, 24, 28, 22, 22, 28, 22, 23, 27, 30, 24…
```

```
head(loanData)
```

```
##   isLoanDefault loanAmount interestRate creditGrade employmentYears homeLiving
## 1             0       5000        10.65           B               10       RENT
## 2             0       2400           NA           C               25       RENT
## 3             0      10000        13.49           C               13       RENT
## 4             0       5000           NA           A                3       RENT
## 5             0       3000           NA           E                9       RENT
## 6             0      12000        12.69           B               11        OWN
##   incomeAnnual ageYears
## 1        24000       33
## 2        12252       31
## 3        49200       24
## 4        36000       39
## 5        48000       24
## 6        75000       28
```

```
loanDataNoOutliers = loanData[!is.na(loanData$incomeAnnual) & !(2500000 < loanData$in
comeAnnual), ]

isMissingInterestRate = is.na(loanDataNoOutliers$interestRate)
isMissingEmploymentYears = is.na(loanDataNoOutliers$employmentYears)

loanDataNoOutliersNA = loanDataNoOutliers

loanDataNoOutliersNA$interestRate[isMissingInterestRate] = median(loanDataNoOutliersN
A$interestRate, na.rm = TRUE)
naInterestRate = as.integer(isMissingInterestRate)
loanDataNoOutliersNA = cbind(loanDataNoOutliersNA, naInterestRate)

loanDataNoOutliersNA$employmentYears[isMissingEmploymentYears] = median(loanDataNoOut
liersNA$employmentYears, na.rm = TRUE)
naEmploymentYears = as.integer(isMissingEmploymentYears)
loanDataNoOutliersNA = cbind(loanDataNoOutliersNA, naEmploymentYears)

set.seed(2020)

loanTestIndices = sample(1:nrow(loanDataNoOutliersNA), nrow(loanDataNoOutliersNA)*1/
3, replace = FALSE)
loanTest = loanDataNoOutliersNA[loanTestIndices, ]
loanTrain = loanDataNoOutliersNA[-loanTestIndices, ]
```

```
getGini = function(good, bad) {
   total = good+bad
   return(2*(bad/total) * (good/total))
}

defaults = sum(loanData$isLoanDefault)
non_defaults = sum(0 == loanData$isLoanDefault)

gini_root = getGini(non_defaults, defaults)
paste("The Gini of the root is:", gini_root)
```

```
## [1] "The Gini of the root is: 0.197239678524086"
```

```
# Left side counts
defaults_left = 1000
nondefaults_left = 8015
gini_left = getGini(nondefaults_left, defaults_left)

# Right side counts
defaults_right = 2227
nondefaults_right = 17850
gini_right = getGini(nondefaults_right, defaults_right)

# Total counts
total_left = defaults_left + nondefaults_left
total_right = defaults_right + nondefaults_right
total_all = total_left + total_right

# Gini gain
gini_gain = gini_root -
  (total_left/total_all * gini_left) -
  (total_right/total_all * gini_right)

gini_left
```

```
## [1] 0.1972432
```

```
gini_right
```

```
## [1] 0.1972381
```

```
gini_gain
```

```
## [1] 4.622219e-12
```

```
# The gini_gain is essentially zero, indicating no value in purchasing feature xyz. T
he near-zero gain means the feature does not improve the purity of the split, suggest
ing the data is already relatively balanced or the feature provides no useful discrim
inatory power.
```

```
library(rpart)
```

```
tree_defaults = rpart(isLoanDefault ~ ., method="class", data = loanTrain)
```

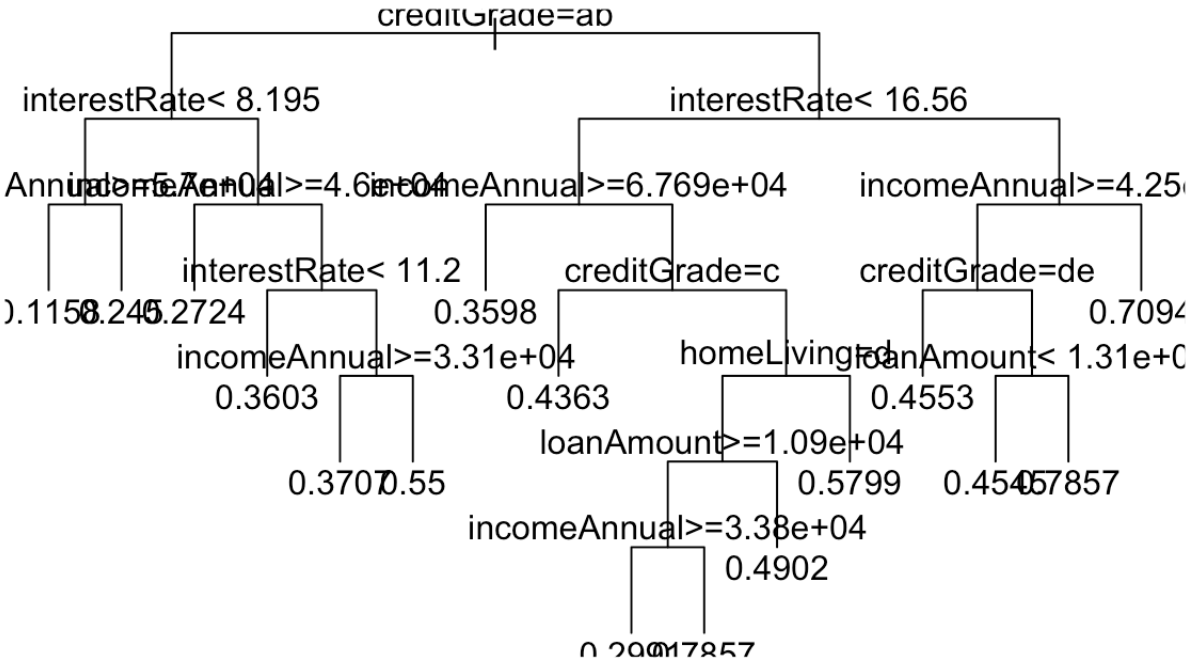
```
# plot(tree_defaults) - commented out; produces an error because the tree has no spli
ts
```

```
# Error: fit is not a tree, just a root
# The decision tree produced no splits because the dataset is heavily imbalanced. Wit
h very few defaults relative to non-defaults, the gini gain from any split is near ze
ro, so rpart creates only a root node with no branches.
```

```
# Select all rows where the loan defaulted
loanTrain_default = loanData[1 == loanData$isLoanDefault, ]

# Select all rows where the loan did not default
loanTrain_nodefault = loanData[0 == loanData$isLoanDefault, ]

# Randomly sample twice as many non-default cases as there are default cases to creat
e a 2:1 ratio
loanTestIndices = sample(1:nrow(loanTrain_nodefault), 2*nrow(loanTrain_default), repl
ace = FALSE)

# Keep only the sampled non-default rows
loanTrain_nodefault = loanTrain_nodefault[loanTestIndices,]

# Combine default and sampled non-default cases into one balanced training dataset
loanTrain_balanced = rbind(loanTrain_default, loanTrain_nodefault)
```

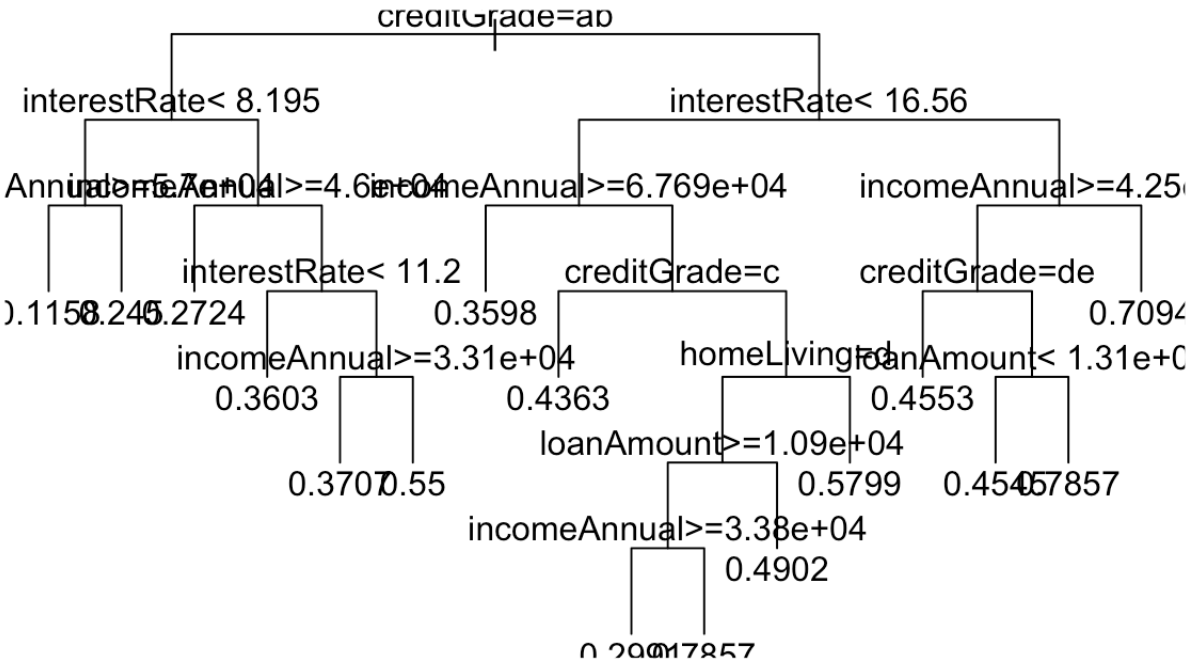
```
library(rpart)
```

```
tree_balanced = rpart(isLoanDefault ~ .,
                      data = loanTrain_balanced,
                      control = rpart.control(cp = 0.001))

plot(tree_balanced, uniform = TRUE)
text(tree_balanced)
```

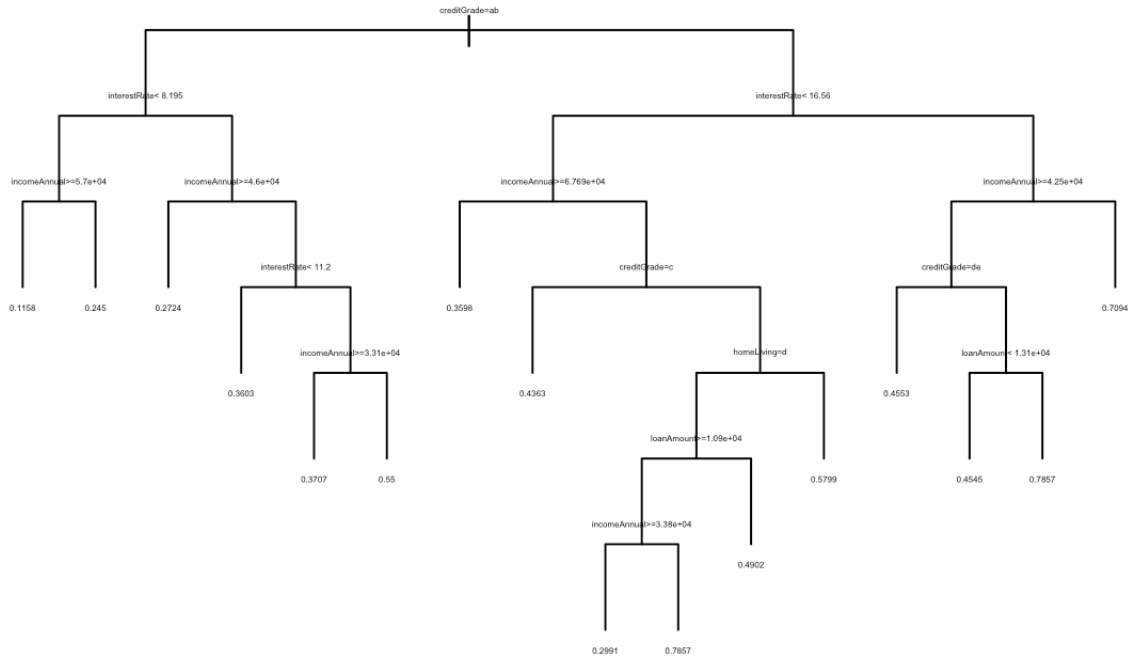


 # uniform = TRUE creates equal vertical spacing between branches, producing a cleaner and more readable decision tree layout.

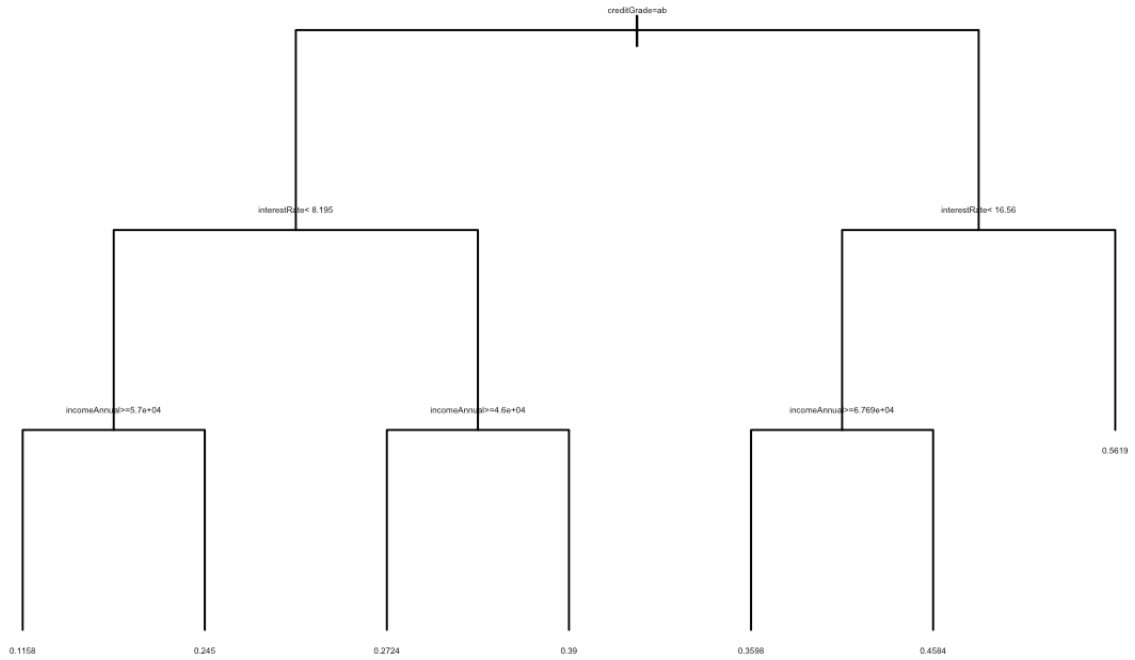
```
tree_balanced = rpart(isLoanDefault ~ .,
                      data = loanTrain_balanced,
                      control = rpart.control(cp = 0.001),
                      parms = list(prior = c(0.7, 0.3)))

plot(tree_balanced, uniform = TRUE)
text(tree_balanced)
```




```
# M = 10 means a missed default (false negative) is penalized 10x more than a false p
ositive
M = 10

tree_defaults_balanced_parms_lossmatrix = rpart(
  isLoanDefault ~ .,
  data = loanTrain_balanced,
  control = rpart.control(cp = 0.001),
  parms = list(loss = matrix(c(0, M, 1, 0), ncol = 2), prior = c(0.67, 0.33)))

plot(tree_balanced, uniform = TRUE)
text(tree_defaults_balanced_parms_lossmatrix, cex = 0.25)
```




```
# Prune using cp = 0.002 (midpoint of the 0.001 to 0.004 range)
tree_lovely = prune(tree_defaults_balanced_parms_lossmatrix, cp = 0.002)

plot(tree_lovely, uniform = TRUE)
text(tree_lovely, cex = 0.25)
```


creditGrade=ab
interestRate< 8.195
interestRate< 16.56
incomeAnnual>=5.7e+04
incomeAnnual>=4.6e+04
incomeAnnual>=6.769e+04
0.5619
0.1158
0.245
0.2724
0.39
0.3598
0.4584